

TÀI LIỆU TRIỂN KHAI KỸ THUẬT CHI TIẾT

Hệ thống Tự động Đánh giá Mã nguồn GitHub dựa trên Trí tuệ Nhân tạo (AI GitHub Grader)

Dự án	AI GitHub Grader (Hệ thống Chấm điểm Mã nguồn Tự động)
Kiến trúc	Enterprise Clean Architecture / Modular Domain-Driven Layer
Phiên bản	v1.0.0 (Môi trường Production / Sẵn sàng Mở rộng)
Ngôn ngữ	Python 3.10+ & Streamlit Framework

1. Tổng quan Hệ thống & Định nghĩa Kỹ thuật

Hệ thống AI GitHub Grader là một giải pháp Enterprise-Grade thuộc phân lớp Automated Code Reviewer & Evaluation System. Mục tiêu cốt lõi của hệ thống là tự động hóa toàn vẹn quy trình kiểm tra mã nguồn từ các kho lưu trữ từ xa (GitHub), bóc tách cấu trúc logic, đối chiếu với tài liệu yêu cầu (Đề bài) và ma trận trọng số (Tiêu chí chấm điểm) để xuất ra báo cáo định lượng chính xác trên thang điểm 100.

Hệ thống được thiết kế theo nguyên lý bền vững, cô lập tối đa tầng giao diện hiển thị (UI) với tầng xử lý tác vụ lõi (Business Logic Services), cho phép doanh nghiệp hoặc cơ sở giáo dục vận hành với độ ổn định cao, bảo mật thông tin mã nguồn và dễ dàng hoán đổi nhà cung cấp mô hình ngôn ngữ lớn (LLM Vendor) mà không gây phá vỡ kiến trúc nền tảng.

2. Kiến trúc Cấu trúc Dự án (Architectural Blueprint)

Dự án tuân thủ nghiêm ngặt mô hình Multi-layered Modular Architecture nhằm loại bỏ hoàn toàn hiện tượng chồng chéo mã nguồn (Spaghetti Code) và liên kết cứng (Hard-coding). Toàn bộ luồng dữ liệu di chuyển theo một chiều từ ngoài vào trong.

```
ai-github-grader/
├── .env                # Lưu trữ các tham số cấu hình bảo mật cục bộ
├── requirements.txt    # Định nghĩa các thư viện phụ thuộc của hệ thống
├── app.py              # Điểm khởi chạy ứng dụng (Presentation Layer)
├── config/             # Tầng cấu hình và quản trị tài nguyên hệ thống
│   ├── __init__.py
│   └── settings.py    # Điểm quản lý biến môi trường tập trung
├── services/           # Tầng nghiệp vụ cốt lõi (Core Service Layer)
│   ├── __init__.py
│   ├── github_service.py # Engine bóc tách, quét đệ quy cấu trúc cây thư mục Git
│   └── ai_service.py    # Engine xử lý ngữ cảnh Prompt và điều phối Mô hình AI
```

3. Chi tiết Mã nguồn Triển khai Hệ thống

3.1. Tầng Cấu hình Hệ thống (System Configuration Layer) - config/settings.py

```
import os
from dotenv import load_dotenv

load_dotenv()

class Settings:
    GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "")
    GITHUB_TOKEN = os.getenv("GITHUB_TOKEN", "")
    DEFAULT_MODEL = os.getenv("DEFAULT_MODEL", "gemini-1.5-pro")

    ALLOWED_EXTENSIONS = ('.py', '.java', '.js', '.ts', '.cpp', '.c', '.cs', '.html',
'.css', '.go', '.kt', '.php')
    EXCLUDED_DIRS = ('node_modules/', '.venv', 'env/', '.git/', 'build/', 'dist/',
'target/', '__pycache__/', '.idea/', '.vscode/')

    @classmethod
    def validate(cls):
        if not cls.GEMINI_API_KEY:
            raise ValueError("CRITICAL CONFIG ERROR: Thieu GEMINI_API_KEY trong .env")
```

3.2. Engine Phân tích Kho lưu trữ Git - services/github_service.py

```
import requests
from config.settings import Settings

class GitHubService:
    def __init__(self):
        self.headers = {"Accept": "application/vnd.github.v3+json"}
        if Settings.GITHUB_TOKEN:
            self.headers["Authorization"] = f"token {Settings.GITHUB_TOKEN}"

    def _parse_url(self, repo_url: str) -> tuple:
        try:
            clean_url = repo_url.strip().rstrip("/")
            parts = clean_url.replace("https://github.com/", "").split("/")
            if len(parts) < 2:
                raise ValueError()
            return parts[0], parts[1]
        except Exception:
            raise ValueError("Duong dan sai. Dinh dang chuan: https://github.com/user/repo-name")

    def get_repo_contents(self, repo_url: str) -> dict:
        username, repo = self._parse_url(repo_url)
        target_branches = ['main', 'master']
        tree_data = None
        detected_branch = 'main'

        for branch in target_branches:
            api_url = f"https://api.github.com/repos/{username}/{repo}/git/trees/{branch}?recursive=1"
```

```

        response = requests.get(api_url, headers=self.headers)
        if response.status_code == 200:
            tree_data = response.json().get("tree", [])
            detected_branch = branch
            break

    if not tree_data:
        raise RuntimeError("Khong the truy xuat du lieu tu GitHub Tree API.")

    code_payload = ""
    processed_files = 0

    for item in tree_data:
        if item["type"] == "blob" and
item["path"].endswith(Settings.ALLOWED_EXTENSIONS):
            if any(ex_dir in item["path"] for ex_dir in Settings.EXCLUDED_DIRS):
                continue

            raw_url = f"https://raw.githubusercontent.com/{username}/{repo}/refs/heads/{detected_branch}/{item['path']}"
            file_response = requests.get(raw_url, headers=self.headers)

            if file_response.status_code == 200:
                code_payload += f"\n\n=====\\n"
                code_payload += f"FILE PATH: {item['path']}\\n"
                code_payload += f"=====\\n"
                code_payload += file_response.text
                processed_files += 1

    if processed_files == 0:
        raise FileNotFoundError("Khong tim thay tep ma nguon phu hop.")

    return {"total_files": processed_files, "content": code_payload}

```

3.3. Engine Xử lý Trí tuệ Nhân tạo - services/ai_service.py

```

import google.generativeai as genai
from config.settings import Settings

class AIService:
    def __init__(self):
        Settings.validate()
        genai.configure(api_key=Settings.GEMINI_API_KEY)
        self.model = genai.GenerativeModel(Settings.DEFAULT_MODEL)

    def generate_grading_report(self, assignment: str, criteria: str, code_content: str) -> str:
        structured_prompt = f"Review code based on assignment: {assignment}, criteria: {criteria}, content: {code_content}"
        response = self.model.generate_content(structured_prompt)
        return response.text

```

3.4. Giao diện Người dùng Tầng Hiển thị - app.py

```
import streamlit as st
from services.github_service import GitHubService
from services.ai_service import AIService

def main():
    st.set_page_config(page_title="AI GitHub Grader v1.0", page_icon="🤖", layout="wide")
    st.title("🤖 Hệ thống AI Khảo Thi & Chấm Điểm Mã Nguồn")
    st.markdown("---")

    col_config, col_monitor = st.columns([2, 3])

    with col_config:
        st.subheader("📋 Phạm vi Thiết lập")
        repo_url = st.text_input("🔗 URL GitHub Repository:", placeholder="https://github.com/user/repo-name")
        assignment_input = st.text_area("📝 Đề bài bài tập:", height=150)
        standard_criteria = "Criteria here"
        criteria_input = st.text_area("🎯 Tiêu chí chấm điểm (Tổng 100):",
        value=standard_criteria, height=150)
        execute_trigger = st.button("🚀 Thực thi Chấm điểm", type="primary",
        use_container_width=True)

    with col_monitor:
        st.subheader("📊 Báo cáo Giám định từ AI")
        if execute_trigger:
            if not repo_url or not assignment_input or not criteria_input:
                st.error("Vui lòng điền đầy đủ các thông tin bắt buộc.")
                return
            try:
                with st.spinner("🔄 Đang lấy mã nguồn từ GitHub API..."):
                    github_engine = GitHubService()
                    extracted_payload = github_engine.get_repo_contents(repo_url)
                    st.success(f"Nạp mã nguồn thành công {extracted_payload['total_files']} tập tin.")

                with st.spinner("🤖 Trợ giảng AI đang đánh giá logic..."):
                    ai_engine = AIService()
                    final_report = ai_engine.generate_grading_report(assignment_input,
                    criteria_input, extracted_payload["content"])
                    st.markdown(final_report)
            except Exception as e:
                st.error(f"Lỗi vận hành: {str(e)}")

if __name__ == "__main__":
    main()
```

4. Ma trận Đánh giá Kỹ thuật (Trade-off Matrix)

Chỉ số kỹ thuật	Lợi ích mang lại (Benefits)	Rủi ro & Giải pháp giảm thiểu (Risks & Mitigations)
Tính Mở rộng Scalability	Kiến trúc module giúp dễ dàng thay thế giao diện hiển thị hoặc chuyển đổi nhà cung cấp LLM (OpenAI, Claude) mà không phá vỡ logic cũ.	Tài nguyên mã nguồn dự án quá lớn dễ gây tràn Token. <i>Giải pháp:</i> Lọc loại bỏ triệt để các thư mục chứa mã nhị phân/thư viện rác tại tệp Settings.
Độ Bảo mật Security	Toàn bộ khóa bí mật và token xác thực được cô lập tại tệp cấu hình môi trường hệ thống, ngăn chặn rò rỉ mã nguồn.	Gửi dữ liệu mã nguồn độc quyền qua API bên thứ ba. <i>Giải pháp:</i> Sử dụng các điều khoản thương mại Enterprise của Google để chặn việc dùng dữ liệu để huấn luyện lại LLM.

5. Hướng dẫn Triển khai Từng bước (Deployment Runbook)

Bước 1: Tạo không gian làm việc và kích hoạt phân vùng môi trường ảo độc lập:

```
mkdir ai-github-grader && cd ai-github-grader
python3 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
```

Bước 2: Tạo cấu trúc cây thư mục chuẩn doanh nghiệp:

```
mkdir config services
touch .env requirements.txt app.py config/__init__.py config/settings.py services/
__init__.py services/github_service.py services/ai_service.py
```

Bước 3: Khai báo các khóa bí mật vào tệp tin .env và cài đặt thư viện:

```
# Điền dữ liệu thật vào .env: GEMINI_API_KEY=AiZaSy... và GITHUB_TOKEN=ghp...
pip install -r requirements.txt
streamlit run app.py
```